TatvaAI: build guide for two people with Claude Code

Source: `PramaanAI_SIH26261_Team_Guide.pdf` (SIH26261, AICTE Regulation Bureau). This file turns it into what you build, with what, and in what order. The old code is archived in `../_old/tatvaai` and can be copied from when useful.

Core idea: a college uploads its EoA documents. The system extracts fields, verifies documents against (mock) issuers, checks them cryptographically and forensically, checks consistency and Handbook norms, then gives a risk tier (GREEN no risk, AMBER medium, RED high) and a ranked inspection list. Every finding is evidence-linked. An officer decides. **The software never approves or rejects.**

1. Feature list (what the prototype has)

Tiers: **P0** = demo dies without it. **P1** = makes the demo strong. **P2** = only if time is left.

A. Application and documents (institution side)

#	Feature	Tier
A1	Login with 3 roles: institution, officer, inspector (JWT, bcrypt, role check on every route)	P0
A2	Create application (academic year + category: EoA, increase in intake, name change)	P0
A3	Per-category checklist of required documents (from YAML)	P0
A4	Upload PDF/JPG/PNG/CSV: type/size/magic-byte check, SHA-256, stored	P0
A5	Auto-classify the document and say it back ("recognised as FIRE SAFETY CERTIFICATE, 97%")	P0
A6	Pre-check: plain-language fix-list (Must fix / Should fix / Minor) with "how to fix" and Handbook clause	P0
A7	Submit locks the application; officer can reopen once	P1
A8	Pre-fill from last year's application	P2

B. Document intelligence

#	Feature	Tier
B1	Text-layer extraction (PyMuPDF) with **page + bounding box + confidence** for every field	P0
B2	Digital signature validation (pyHanko): UNSIGNED / SIGNED_VALID / SIGNED_MODIFIED / SIGNED_INVALID / SIGNED_UNTRUSTED	P0
B3	"What changed after signing": cut the PDF back to the signed revision and diff the field	P1
B4	QR decode (pyzbar, OpenCV fallback) and compare with printed text	P0
B5	Photo EXIF: GPS, capture time, camera; perceptual hash	P0
B6	Faculty CSV into faculty records	P0

#	Feature	Tier
B7	OCR for scans (PaddleOCR), behind a flag	P2

C. Verification with issuers (all MOCK)

#	Feature	Tier
C1	One adapter per source: e-stamp (SHCIL), DigiLocker, ICAI UDIN, state fire NOC, state occupancy, AICTE records, accreditation list	P0
C2	Six statuses: VERIFIED, MISMATCH_DETECTED, EXPIRED, VERIFICATION_PENDING, SOURCE_UNAVAILABLE, MANUAL_REVIEW_REQUIRED	P0
C3	Every check stores source, timestamp, request hash, response hash, summary	P0
C4	Source down routes to manual review; officer records a manual confirmation with a note	P0
C5	Carry-forward: same SHA-256 verified last year is reused as VERIFIED	P1

D. The five engines (each produces flags)

Each flag has: code, engine, severity (CRITICAL/MAJOR/MINOR/INFO), confidence 0 to 1, message, explanation, evidence list, and for rule flags the rule id, rule-pack version and APH clause and page. Forensic flags are always labelled "Preliminary indicator, requires human verification".

Engine	Flag codes	Tier
Issuer	ISSUER_MISMATCH, ISSUER_EXPIRED, ISSUER_UNAVAILABLE	P0
Authenticity	SIGNATURE_INVALID, MODIFIED_AFTER_SIGNING, QR_MISMATCH, TAMPER_SUSPECTED, METADATA_ANOMALY, DUPLICATE_DOCUMENT_CROSS_INSTITUTION	P0 (ELA heatmap and font checks P1)
Completeness	DOC_MISSING, FIELD_MISSING, DOC_UNSIGNED, DOC_EXPIRED, DOC_EXPIRES_BEFORE_AY, PROVISIONAL_OC_LIMIT_EXCEEDED	P0
Consistency	MISMATCH_CROSS_DOC, MISMATCH_CROSS_YEAR, DISCLOSURE_MISMATCH (website vs application), TIMELINE_IMPOSSIBLE, ACCREDITATION_CLAIM_UNVERIFIED	P0 (website and cross-year P1)
Compliance and infrastructure	NORM_FACULTY_SHORTFALL, NORM_CADRE_RATIO, NORM_ROOMS, NORM_AREA, NORM_PC_RATIO, NORM_BANDWIDTH, DUPLICATE_FACULTY_CROSS_INSTITUTION, DUPLICATE_PHOTO, PHOTO_GPS_OFFSITE, BUILTUP_AREA_IMPLAUSIBLE, BUILDING_NOT_VISIBLE_ON_DATE	P0 for NORM_*, P1 for photo/geo checks

E. Evidence, rules, scoring

#	Feature	Tier
E1	Evidence graph: facts (subject, attribute, value, unit, source) with provenance document/form/system/derived	P0
E2	**Rule Pack**: Handbook norms as versioned YAML with clause and page; safe evaluation with `simpleeval`; PASS / FAIL / INSUFFICIENT_EVIDENCE	P0
E3	Risk score and tier with top reasons (weights from the `risk` YAML)	P0
E4	Ranked inspection list (RED first) with "why this institution" and an on-site checklist	P0
E5	Officer rule editor: validate, dry-run (risk now vs under draft), activate	P1

F. Officer and inspector screens

#	Feature	Tier
F1	Review queue: KPI cards, tier chart, filters, sorted RED first	P0
F2	Workbench tabs: Summary, Flags and Evidence, Verification, Compliance	P0
F3	Evidence viewer: PDF page with the field's box highlighted	P0
F4	Accept or dismiss a flag **with a required reason**; no reject button anywhere; persistent banner "Decision support only"	P0
F5	Infrastructure tab: Leaflet map with 1 km radius, photo markers, duplicate photo side by side	P1
F6	Timeline tab, ELA heatmap toggle	P2
F7	Inspector page: ranked list plus checklist PDF export	P1
F8	Alerts bell and nightly job (missing info, expiring in 60 days, expired, pending over 7 days, anomalies)	P1
F9	Validation report PDF with cropped evidence images and a footer SHA-256	P1

G. Safeguards (judges check these)

#	Feature	Tier
G1	Role-based access; institutions see only their own data	P0
G2	Hash-chained audit log with a "verify chain" button	P0
G3	Document access log (who viewed what)	P1
G4	MFA (TOTP) for officer	P2
G5	Synthetic data only; every external system labelled `"mock": true`	P0

2. What the prototype will contain (deliverables)

1. A web app for 4 roles (React).
2. A REST API with OpenAPI docs at `/docs` (FastAPI).
3. A **mock-issuers service** (separate small FastAPI app) that pretends to be SHCIL, DigiLocker, UDIN, fire and occupancy portals, AICTE records, and institution websites.
4. A **synthetic dataset** generator: 3 institutions that tell the whole story. A is clean (GREEN), B has two defects (AMBER), C has about 12 planted problems (RED). Plus last year's data for cross-year checks. Real signed PDFs come from a **test PKI** (`make_test_pki`), so signature validation is genuine.
5. The **Rule Pack** (YAML): documents, norms, timelines, categories, risk.
6. Two proof scripts: `verify_planted` (every planted problem is caught, no false alarms) and `evaluate` (extraction precision/recall, anomaly recall, tier accuracy, seconds per application). These are your numbers for the pitch.
7. Tests, a 5-minute demo script and a judge Q&A sheet.

3. Tech stack

Keep it as small as it can be while still telling the integration story.

Layer	Choice	Why / simplification
Frontend	React 18 + TypeScript + Vite, React Router, `react-pdf` (PDF.js), Leaflet, Recharts, Tailwind	Highlighting a bounding box on a PDF page is the signature feature; `react-pdf` does it
Backend	Python 3.11, FastAPI, Pydantic v2, SQLAlchemy 2, Alembic	Auto OpenAPI means the frontend can be generated from it
Database	**SQLite first**, same models run on PostgreSQL 16 later	Skip PostGIS (lat/lon columns are enough)
Background jobs	**None at first**: run the pipeline synchronously (2 to 5 s). Add Celery + Redis only for the nightly alert job, P1	One less service for two people to debug
File storage	Local disk (`STORAGE=local`)	MinIO only if asked
PDF read	PyMuPDF (text with boxes, page render at 150 dpi)	
Signatures	pyHanko	Real validation, not a fake flag
QR	pyzbar, OpenCV fallback	
Images	Pillow (EXIF, ELA), `imagehash` (perceptual hash)	
Rules	YAML + `simpleeval`	Norms are config, not code
Auth	JWT + bcrypt, `pyotp` for TOTP	
Reports	ReportLab (PDF)	
Mock issuers	Separate FastAPI app, port 9000	Adapter base URLs come from env, so real APIs can replace mocks
Optional, flags off	PaddleOCR, YOLO, local Ollama for drafting rules	P2 only
Dev	uv or venv, pnpm, pytest, vitest, Docker Compose at the end	Do Docker last; the old build never ran it end to end, so test it early this time

Ports: frontend 5173, backend 8000, mock issuers 9000.

4. Integration plan (so it never becomes a problem)

Integration breaks when two people guess each other's interfaces. Fix the interfaces first, then work in parallel.

Step 0: freeze four contracts (do this together, about half a day, commit them before any feature code):

1. `rulepacks/schema.json` and the rule YAML layout.
2. DB models + first Alembic migration (19 tables, see the guide's section 6.5).
3. `openapi.json` skeleton: the 10 routers (`auth, mfa, applications, documents, precheck, verification, rules, risk, alerts, audit`) with request/response models, returning stub data.
4. The flag object and the fact object (shapes in section 7 and 6.4 of the guide). These two are the glue between every engine and every screen.

Rules of the road

- Backend person owns `backend/`, `mock_issuers/`, `rulepacks/`, `scripts/`. Frontend person owns `frontend/`. Nobody edits the other's folder without telling them.
- The frontend develops against the stub API from day 1; swap to the real one without changing screens.
- Regenerate the TypeScript client from `openapi.json` (`openapi-typescript`) whenever the backend contract changes. A contract change is its own commit with a message starting `contract:`.
- Pipeline engines are pure functions: `engine(context) -> list[Flag]`. That makes each engine testable alone and lets either person add one.
- Every external source sits behind an adapter with `base_url` from env. Nothing else knows it is a mock.
- One `main` branch protected by tests. Short feature branches, merge daily.

Work split

Person 1: Backend and data	Person 2: Frontend and demo
DB, auth, upload, extraction, signature, QR	Shell, login, role menus, banners
Mock issuers + adapters	Institution pages: applications, upload, pre-check fix-list
Evidence graph, rule pack loader, the 5 engines	Officer queue and workbench (4 tabs first)
Risk scoring, inspection list	Evidence viewer with highlighted box
Synthetic generator, test PKI, `verify_planted`, `evaluate`	Inspector page, officer audit/rules, map
Report PDFs, audit hash chain	Demo script, slides, Q&A, UI polish

5. Working with Claude Code (two Pro accounts)

- Put a short `CLAUDE.md` at the repo root: stack, folder ownership, the four frozen contracts, "never add a reject button", "everything is synthetic and MOCK", how to run tests.
- Keep a `DECISIONS.md`: one line per decision plus the reason. Both Claudes read it first; it stops the two of you drifting.
- Give each Claude **one task with a check** ("write the completeness engine; `pytest tests/test_completeness.py` must pass"). Ask for tests before code on engines; the planted problems are the spec.
- Use `git worktree` or separate clones so two sessions never edit the same files.
- Pro limits are real: do the heavy generation (synthetic data, boilerplate, tests) early, keep sessions short and scoped, and don't paste whole PDFs into chat. Point Claude at `docs/` files instead.
- Commit after every green test run.

6. Build order

Phase	Goal	Exit check
0 Contracts	Repo, `CLAUDE.md`, frozen contracts, stub API running, frontend login against stubs	Both can run the app and see stub data
1 Data	Test PKI, synthetic generator for A, B, C, `expected_flags.json` (ground truth)	41 documents generated and deterministic

Phase	Goal	Exit check
2 Ingestion + intelligence	Upload, classify, extract with boxes, signature, QR, EXIF	All ground-truth fields extracted
3 Evidence + rules + engines	Evidence graph, rule pack, mock issuers, five engines, flags saved	`verify_planted` catches every planted problem
4 Scoring + screens	Risk tier, queue, workbench, evidence viewer, pre-check	A GREEN, B AMBER, C RED end to end
5 Safeguards + extras	Audit chain, decisions with reasons, inspection list, PDF report, alerts, map	Demo script runs clean
6 Proof + polish	`evaluate`, tests, Docker compose, demo rehearsal	Numbers in a table for the pitch

Cut from the bottom if time runs short: P2 first, then F5/F6, then alerts. Never cut P0 or the audit chain.

7. Do's and don'ts for the pitch

- Say: decision support, evidence-linked, preliminary indicator, verify once and reuse every year, Handbook as code.
- Don't say: auto-approve, auto-reject, fraud detected, catches all forgeries.
- Be honest that all metrics are on synthetic data and all issuers are mocks. The guide's own section 16 lists the limits; repeat them before a judge finds them.
- Check every Handbook clause against the official APH PDF before quoting it (printed page N = PDF page N + 16).